Corrections required in the Review #2 deck and the LLD

The repository itself is now consistent: every script reads one dataset, and every number is generated by `make results` into `results/CANONICAL.md`. The errors below exist only in the **external** documents (the PPT and the LLD PDF), which are not in this repo and so could not be fixed from here.

Verified against code and data on 2026-10-12. Each row cites where the true value comes from.

Must fix — an examiner who checks will find these

#	Document says	Truth	Source
1	**7** EC2 instance types (PPT)	**5** — c5a, c7i, m5a, r5n, z1d	`canonical_metrics.json → dataset.machines`; the dataset has exactly 5 `machine_id` values
2	Classifier macro-F1 **0.562** (LLD)	**0.375**	`CANONICAL.md §5`. 0.562 was computed on the buffer-bug dataset; `results/bottleneck_classifier_corrected.json` holds both side by side
3	Classifier presented as working	`io` **recall = 0.000** — it never once predicts the `io` class correctly	`CANONICAL.md §5` confusion matrix
4	Analytical MAPE **31.14%** (older pasted run)	**21.61%**	That run used the **single-core** compute signal. With it, analytical loses outright. The 21.61% requires the **multi-core** signal. Always state which.
5	`cetp.*` package names, `QueryProfiler`, `ConfidenceGate`, `MeasurementStore` etc. as a class inventory	**No such packages or classes exist.** The logic is real but lives in flat scripts	See the mapping table below
6	"Empirical percentile margins m_95/m_99"	**Not implemented anywhere.** Remove the claim or implement it	`grep -r "m_95\`

Must add — true, and in your favour

#	Claim to add	Source
7	Bootstrap CI on the improvement is **[0.32, 2.56] pp**, excludes zero, recomputed on the corrected dataset with a leakage-free classifier	`CANONICAL.md` headline
8	The parallelism mechanism is **causally confirmed**, not hypothesised: r = 0.458, p = 0.037	`results/fix3_parallelism_mechanism_summary.csv`
9	The analytical formula's advantage is **protocol-invariant** (21.61 / 21.66 / 21.64 across LOMO / LOQO / LOMQO) while the learned model swings from 17.29% to 26.86% depending purely on whether it has seen the target hardware	`CANONICAL.md §1`
10	The analytical gain is **localised**: on `compute`-labelled queries it is 22.63% → 17.55%; on `bandwidth` and `io` queries it is identical to the baseline by construction	`CANONICAL.md §4`
11	**Repetition 1 of every cell is a cold-cache run** (slowest of 20 in 61/105 cells, up to 8.33x). p50 is unaffected; **p95/p99 published before this audit are contaminated**	`CANONICAL.md §6`
12	42 of 105 cells exceeded the protocol's own 5% CV threshold before warmup discard; 11 after	`CANONICAL.md §6`

Must soften — claims the evidence does not carry

#	Do not say	Say instead
13	"The CI/CD gate blocks SLA violations"	It blocks ~8% of pairs with a **54.5% false-alarm rate**, and the PASS state covered 4 of 840 pairs. It is a prioritisation signal, not an automatic stop.
14	"Quantile regressors predict p95/p99"	They under-cover badly (80.7% empirical at nominal 95%) **and** those targets are warmup-contaminated. Regenerate before claiming anything about tails.
15	"The classifier identifies the bottleneck"	The 4 labels are **21 hand-assigned judgement calls** (Q2 and Q13 acknowledged coin flips), and because all six classifier features are machine-invariant it is effectively a 21-row lookup table.
16	"The system generalises across hardware"	Across **5** EBS-backed instance types at **SF1** only, single-query-at-a-time. One outlier machine (c5a) demonstrates the extrapolation failure mode exists; n=1 cannot say how often it occurs.
17	~~"The gate uses the validated hardware signature"~~	**FIXED 2026-10-13.** Both `models/hardware_signature.pkl` and `gate/cetp.yml` now carry the multi-core scores, and `canonical_evaluation.py` verifies that at runtime instead of asserting it. The gate does embody the headline finding.

Superseded entirely — row 5 and the "class inventory" problem

Row 5 said no `cetp.*` packages existed. **They exist now**: `cetp/` is an installable package (`predict`, `regress`, `redact`, `signature`, `cross_env`, `operator_model`, `plan_nodes`, `calibration_store`, `ingest`, `telemetry`, `cli`). A rewritten LLD should document the real module list rather than either the old fiction or the old flat scripts.

The LLD and deck are not in this repository, so they still cannot be edited from here. What has been done instead is to write a correct one from scratch: `docs/LLD.md` and its PDF, generated entirely from verified numbers in `results/`. Treat the old PDF as superseded rather than patching it row by row — every headline figure in it predates the buffer-bug correction, the multi-core fix, and the per-operator work.

Design-vocabulary mapping

The LLD's class names were written as a design view, not from the code. Present them as this mapping table rather than as an inventory of classes that exist.

LLD name	Where the logic actually lives
`QueryProfiler` (root-node buffers)	`root_buffers()` + `features_from_explain_json()`, `gate/cetp_gate.py:27–42`
`HardwareProbe`	`scripts/collection/compute_probe.c`, `run_compute_benchmark.sh`
`HardwareSignature`, `ratioTo`, `distanceTo`	`models/hardware_signature.pkl` (a dict) + `hardware_distance_confidence()`, `gate/cetp_gate.py:102`
`WorkloadCostVector`	the `dev_*` columns built in `build_pairs()`
`BottleneckClassifier`	`fit_classifier()` / `phase5_classifier.py`
`QuantileRegressor`	`fit_quantile()` / `phase6_scaling_regressor.py`
`AnalyticalEstimator`	`analytical_scaling()`, `gate/cetp_gate.py:68`
`ConfidenceGate`	`hardware_distance_confidence()`, `gate/cetp_gate.py:102`
`MeasurementStore`, `persist`, `registerMachine`, `triggerRetrain`	`scripts/analysis/measurement_store.py` — `record_measurement()`, `retrain()`, SQLite with `machines` / `measurements` / `retrain_log`. **There is no `captured_runs` table**; the real table is `measurements` with a `source` column.
`RuntimePredictor`	`predict_scaling()`, `gate/cetp_gate.py:78`
`CICDGate`	`three_state_verdict()` + `run_gate()`, `gate/cetp_gate.py:120–191`

Confirmed as implemented — safe to claim

- **Root-node-only buffer capture.** `root_buffers()` reads the root node's cumulative counts. Summing per-node was the bug; it is fixed in the gate and corrected in the dataset.
- **Hardware signature** = STREAM-triad bandwidth + OpenMP multi-core compute, stored per machine.
- **4-class probability output** feeding the regressor as `p_compute/p_bandwidth/p_io/p_mixed`.
- **Quantile regressors** for p50/p95/p99 with `random_state=42`.
- **Analytical estimator exists and acts as a fallback.**
- **Confidence gate semantics**: `confidence = max(0, 1 − min_distance / √2)` over normalised (bandwidth, compute) space, threshold 0.6. The 0.6 is a hand-set constant, not derived — say so.
- **Analytical weights**: mixed = `sqrt(inv_bandwidth × inv_compute)` (geometric mean), io term = `1/bandwidth_ratio`. Both confirmed at `gate/cetp_gate.py:68–71`. Note the io weight is folded in with bandwidth, which is *why* §4 shows no io improvement.
- **CI/CD gate** returns PASS / BLOCK / WARN, with WARN also on low confidence. `.github/workflows/` runs it on q19, q1, q18 on every push. 24 tests pass.
- **Self-calibration loop**: persist → register machine → retrain, with versioned artifacts in `models_online/v2/`.
- **13 features, no `prod_*` leakage** in the feature list.

- **Model registry files exist**: `models/bottleneck_classifier.pkl`, `models/scaling_quantile_models.pkl`, `models/hardware_signature.pkl`. Note the names differ from the LLD's `quantile_regressor.pkl` / `bottleneck_clf.pkl` / `hardware_signatures.json`.

Yours to fix, not code issues

Appendix C dates, Appendix D HLD section numbers, and the guide designation.